Advanced Data Structures

Lecture 1 – Course Overview & Foundations

Dr. Suraj Kumar

Department of Computer Science & Engineering

28 July 2026

About this Course

Goal

Pair every structure with the mathematics that guarantees its performance – not just *how* it works, but *why* it is fast.

Learning outcomes

- **Design.** Choose and build the right structure.
- **Analyse.** Prove worst-case and amortized bounds.
- **Apply.** Map structures to real systems and workloads.

Prerequisites

- Data Structures: stacks, queues, BSTs, hashing
- Asymptotic notation and recurrences
- Discrete mathematics: proofs, basic probability

Where these appear

- **Databases & filesystems** – balanced trees
- **Compilers & OS** – union–find, tries
- **Search & bioinformatics** – suffix trees
- **Networks & caches** – Bloom filters
- **Schedulers** – Fibonacci heaps

Class Timetable

Section A

Day	Time
Wednesday	11:30 – 12:30
Thursday	10:30 – 11:30
Friday	10:30 – 11:30

Section B

Day	Time
Monday	11:30 – 12:30
Tuesday	11:30 – 12:30
Wednesday	9:30 – 10:30

Assessment & Marking

Grading scheme

Component	Detail	Marks
End-semester examination	comprehensive	50
Mid-semester examination	Units I–III	30
Quizzes	2×5	10
Assignments	2×5	10
Total		**100**

Continuous assessment

The 20 continuous-assessment marks come from two quizzes and two assignments, each worth 5 marks.

Syllabus at a Glance

Unit	Theme	Lectures
I	Amortized complexity & disjoint sets	L1 – L7
II	Balanced trees & mergeable heaps	L8 – L19
III	Optimal BST, dynamic programming & Bloom filters	L20 – L25
IV	Splay trees, tries, suffix & interval trees	L26 – L34
V	Randomized & string-matching algorithms	L35 – L42

Lecture-wise Plan

Lectures	Topic
Unit I – Amortized Complexity & Disjoint Sets	
L1	Course introduction; review of algorithm analysis
L2–L5	Amortized complexity: aggregate, accounting, potential
L6	Augmented stack, table doubling, binary counter
L7	Disjoint Set ADT (Union–Find)
Unit II – Balanced Trees & Mergeable Heaps	
L8	Dictionary ADT and its operations
L9–L12	Red–Black trees: insert, delete, analysis, augmented
L13–L14	Leftist trees and leftist heaps (HBLT / WBLT)
L15–L16	Binomial heaps and operations (insert, delete, merge)
L17–L19	Fibonacci heaps: decrease-key, delete, amortized analysis
Unit III – OBST, Dynamic Programming & Bloom Filters	
L20–L23	OBST: naïve method and the dynamic-programming solution
L24–L25	Bloom filter: significance and amortized improvement

Lecture-wise Plan

Lectures	Topic
Unit IV – Splay Trees, Tries, Suffix & Interval Trees	
L26–L28	Splay trees: operations and amortized analysis
L29–L31	Binary tries; bitwise and compressed tries
L32–L33	Suffix trees and their characteristics
L34	Interval trees and efficient searching
Unit V – Randomized & String-Matching Algorithms	
L35–L36	Skip lists: operations and complexity analysis
L37–L38	Treaps; selection tree and K-way merging
L39–L41	String matching: naïve, finite automata, Rabin–Karp, KMP
L42	Network-flow algorithms

References

[1] E. Horowitz, S. Sahni, D. Mehta. *Fundamentals of Data Structures in C++.* Primary text.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. PHI. Primary text.

[3] M. A. Weiss. *Data Structures and Algorithm Analysis in C++.* Addison-Wesley.

[4] D. P. Mehta, S. Sahni. *Handbook of Data Structures and Applications.*

[5] R. Motwani, P. Raghavan. *Randomized Algorithms.* Advanced reference.

Why Advanced Data Structures?

The same problem can be trivial or hopeless – the structure decides

The wrong structure

- Membership in a big set with a list: $O(n)$ per query
- Repeated min-extraction from an array: $O(n)$ each time

The right structure

- Bloom filter: $O(1)$ with tiny memory
- Fibonacci heap: $O(1)$ amortized decrease-key

Every system you rely on is built on these ideas

- **Databases & filesystems.** B-trees and Red–Black trees index billions of records.
- **Search engines.** Tries and suffix trees handle text at web scale.
- **Operating systems.** Union–Find and interval trees manage resources.
- **Networks & caches.** Bloom filters cut expensive lookups to a single probe.

Reviewing Algorithm Analysis

Cost as a function of input size n, as $n \to \infty$

Definition (asymptotic notation)

- $O(g)$ – upper bound; grows no faster than g
- $\Omega(g)$ – lower bound; grows at least as fast as g
- $\Theta(g)$ – tight bound; same growth rate as g

Constants and machine details are ignored; only the growth rate matters.

Growth rates (slow → fast)

Order	**Name**
$O(1)$	constant
$O(\log n)$	logarithmic
$O(n)$	linear
$O(n \log n)$	linearithmic
$O(n^2)$	quadratic

Which case do we report?

- **Best case** – optimistic and rarely useful.
- **Average case** – requires a probability model of the inputs.
- **Worst case** – our default; it is a guarantee.

But the worst case per operation can badly overstate the cost of a whole sequence – which motivates amortized analysis.

Amortized Complexity: the Core Idea

Definition (amortized cost)

The **amortized cost** of an operation is the guaranteed average cost per operation over a worst-case sequence – *with no probabilistic assumptions* (unlike average-case analysis).

Example: dynamic array (table doubling)

Appending to a full array allocates a $2\times$ array and copies everything, so a single append can cost $O(n)$. Over a whole run of appends, though:

Append #	1	2	3	4	5	6	7	8
cost	1	2	1	4	1	1	1	8

Total copying over n appends:
$1 + 2 + 4 + \cdots + n < 2n = O(n)$.

The upshot

- Total work $= O(n)$
- Amortized per append $= O(1)$

Rare expensive operations are paid for by the many cheap ones.

Amortized $\neq$ average-case: the bound holds for every sequence, assuming nothing about the input.

Three Ways to Prove Amortized Bounds

1. Aggregate method

Bound the total cost of the whole sequence, then divide by the number of operations.

$$\textit{amortized} = T(n)/n$$

2. Accounting method

Charge each operation a fixed price; overcharges are banked as credit that pays for later expensive operations.

$$\textit{banked credit} \geq 0 \ \textit{always}$$

3. Potential method

Define a potential Φ on the data structure; the amortized cost equals the actual cost plus the change in potential.

$$\textit{amortized} = \textit{actual} + \Delta\Phi$$

Common examples

All three methods will be applied to the same running examples: the augmented stack, table doubling, and the binary counter.

Summary

Key points

- The right data structure can change a problem's complexity class.
- Worst-case bounds are our default because they are guarantees.
- Per-operation worst case can be too pessimistic for a sequence.
- Amortized cost = guaranteed average over a sequence, *no probability*.
- Table doubling: $O(n)$ worst-case append, yet $O(1)$ amortized.

Next lecture

The aggregate method in full, applied to the binary counter and the multipop stack.

Reading

- CLRS, Chapter 17 (§17.1–17.2)
- Horowitz & Sahni, amortized complexity